Learning a Battle Simulator from Logged Replays

Machine-learning foundations for a tiered VGC game model

Version 1.0 · Last updated 2026-07-22 Will Hooper · ABRA

> *A research white paper on the modelling problem behind ABRA's stages 2–3 (the simulator and the team optimiser). It states the problem formally, surveys the relevant machine-learning literature, derives the math for each modelling tier, and gives an honest feasibility assessment. Living document; updated in the same pass as the code it describes.*

Abstract

ABRA collects a growing corpus of Pokémon Champions (Reg M-B) battle replays. We want to turn that corpus into a *simulator*: a model that, given two teams, predicts how their game tends to resolve, so that hypothetical teams can be optimised against the live metagame without playing thousands of real games. The underlying game — a two-player, zero-sum, simultaneous-move, stochastic game of imperfect information — is implemented by a closed engine we cannot query. We therefore treat the problem as **learning a model from logged data**. This paper formalises the game, decomposes the simulator into three modelling tiers of increasing fidelity and cost (an outcome model, a hybrid rollout model, and a learned dynamics model), derives the estimator and its failure modes for each, frames team optimisation as the outer loop of a self-improving flywheel, and specifies an evaluation protocol that is honest about the irreducible variance of the domain. We ground each tier in the 2025 Pokémon-AI literature — PokéChamp, Metamon, and VGC-Bench — and conclude with a feasibility assessment: tier 1 is immediately achievable on ABRA's data, tier 2 reuses

the existing CHOMP damage engine, and tier 3 is a genuine research effort that recent work shows to be possible but expensive.

---

1. Introduction

A competitive VGC player faces three nested questions: which six Pokémon to build, which four to bring, and how to play the resulting game. CHOMP answers the middle question with exact damage math. The hardest lever, and the one with the most upside, is the first: *which six beats the current metagame.* Answering it by trial on the ladder is slow and high-variance. The alternative is to **simulate** — to evaluate a candidate team against the meta in software, iterate, and only then play. This paper is about building that simulator from the data ABRA already collects.

The central obstacle is that the Champions battle engine is **closed**. Unlike the standard Smogon formats served by `pokemon-showdown`'s open-source simulator — which the poke-env library exposes to RL agents — the Champions ruleset and its stat system are not available as a queryable environment. We cannot roll the true dynamics forward for a hypothetical state. What we *do* have is a large and growing set of **observed trajectories**: real games, each a sequence of states and joint actions ending in a win or loss. Learning a usable model from such logs, with no ability to interact with the true environment, is exactly the setting of **offline (batch) reinforcement learning** and **model learning / system identification**. This is not a toy analogy: Metamon (§9) reconstructs 5M+ trajectories from human Showdown replays and trains competitive agents on them offline.

2. The game, formally

A single VGC battle is a **partially observable stochastic game (POSG)**, and specifically a two-player, zero-sum, *simultaneous-move* Markov game with imperfect information. We fix notation.

- **State** `s ∈ S`. The full state comprises both teams' species, sets (moves,

item, ability, the Champions SP spread), current HP, status, stat stages, field conditions (weather, terrain, screens, Trick Room), and which Pokémon are active. Level 50; no Tera in Champions.

- **Players** `i ∈ {1, 2}`.
- **Actions**. At each turn both players choose *simultaneously* from a legal set `A_i(s)` — for each of two active Pokémon, a move (with a target) or a switch. The **joint action** is `a = (a_1, a_2)`. Team preview is a special first move: each player chooses an ordered bring of four from six, `A_i = {ordered 4-subsets of the six}`, `|A_i| = 6·5·4·3 = 360`.
- **Transition kernel** `T(s' | s, a)`. Stochastic: damage rolls (16 discrete outcomes, 85–100%), accuracy, secondary-effect procs, critical hits, speed ties. This kernel is the *engine*, and it is what a full simulator must reproduce.
- **Reward**. Zero-sum and terminal: `r = +1` for the winner, `-1` for the loser; `0` elsewhere. Define the **value** of a state to player 1 as `V(s) = E[r | s, play]` under a solution concept.
- **Observation**. Player `i` sees `o_i = O_i(s)`: their own team fully, and of the opponent only what has been revealed (the six species at preview; each move/item/ability only once used or triggered). This partial observability is first-class — it is why opponent modelling is a distinct problem, and why belief states matter.

Two structural facts shape everything downstream:

1. **Simultaneous moves ⇒ mixed strategies.** Because both players commit without seeing the other's choice, the per-turn subgame is a matrix game whose optimal solution is generally a *mixed* (randomised) Nash equilibrium, not a single best move. A simulator that assumes deterministic opponent play is systematically wrong at exactly the decision points that matter.
2. **Imperfect information ⇒ belief states.** The relevant object is not `s` but a distribution over states consistent with the observation history — a belief `b(s)`. Solving the game exactly is the province of **counterfactual regret minimisation (CFR)** and its deep variants;

approximations are unavoidable at VGC's scale.

The team-building question sits one level above the battle: it is a **Bayesian game** in which a player commits to a team before knowing the opponent's, and payoffs are the expected battle values integrated over the opponent distribution the metagame induces.

3. Three modelling tiers

A single "simulator" of full fidelity is neither necessary nor tractable for every use. We decompose it into three tiers, coarse-to-fine, and choose per use (broad search vs. finalist vetting). This is the standard model-based pattern: cheap approximate models for planning breadth, expensive accurate models for depth.

Tier	Object learned	Query	Cost	Primary use
1. JOLTEON (outcome)	`Pψ(win | team_A, team_B)`	one forward pass	~µs (fastest)	search over thousands of teams
2. MEDICHAM (hybrid rollout)	CHOMP damage + learned policies	a few short rollouts	~ms	ranking finalist teams
3. SLOWKING (learned dynamics)	`Tθ(s' | s, a)` + value/policy	belief search / playout	~s (slowest)	deep vetting, a matchup

The rest of the paper derives each.

3.1 The model family, named

Each model gets its own name, in the CHOMP / ABRA tradition — and the Pokémon's *speed* signals the model's cost, fast to slow:

- **JOLTEON** — *Joint Odds, Ladder-Trained Expected-Outcome Network* (Tier 1). The **fastest** model (Jolteon is the quickest of the cast): an instant win-probability call between two teams, from Bradley–Terry strengths plus CHOMP-derived damage/coverage features (§4.3.1). One forward pass, microseconds — fast enough to score a whole team search.
- **MEDICHAM** — *Matchup Evaluation, Damage-Informed CHOMP-Heuristic Approximate Moves* (Tier 2). Middle of the pack, in name and in speed: it plays a matchup out a few turns using **CHOMP's exact damage engine** and behaviour-cloned move priors. Grounded, approximate, mid-cost.
- **SLOWKING** — *Search over Learned Opponent-belief World, Knowledge-Intensive Nash Game-solver* (Tier 3). The **slowest** and deepest: a slow but wise brain that searches over belief states on a learned dynamics model toward equilibrium play. Slow on purpose — you run it only on finalists.
- **DITTO** — *Double-oracle Iterative Team-Tuning Optimiser* (§8, the outer loop). It tries team after team against the meta and transforms toward the strongest six, the way Ditto tries every form.

CHOMP (the bring-4 engine) and the coach are *consumers* of these models, not models themselves. The naming makes the pipeline legible and the speeds honest: ABRA feeds MEDICHAM and JOLTEON; those and SLOWKING feed DITTO; DITTO's teams and SLOWKING's lines reach the player through CHOMP and the coach; the games played feed ABRA. Fast Pokémon do the cheap, broad work; the slow one does the deep work.

4. Tier 1 — JOLTEON, the outcome model

4.1 Objective

Learn a function that maps an ordered pair of teams to a win probability: `Pψ(y = 1 | x_A, x_B)`, where `y = 1` iff team `A` wins and `x` is a feature encoding of a team. This is supervised binary classification on the log: each stored

game contributes one labelled example (and its mirror, with the label flipped, to enforce the antisymmetry `P(A beats B) = 1 − P(B beats A)`).

4.2 The Bradley–Terry / Elo backbone

The natural, well-founded baseline treats each team (or, more usefully, each *component*) as having a latent strength and models paired-comparison outcomes with the **Bradley–Terry** model:

```
P(A beats B) = σ( β_A − β_B ),   σ(z) = 1 / (1 + e^{−z}).
```

Fitting `β` by maximum likelihood over the observed win/loss pairs is exactly logistic regression; online, the same model with a specific update rule is **Elo**. Two refinements matter for VGC:

- **Teams are compositional.** Rather than a strength per *team* (there are astronomically many), parameterise strength as a sum over the team's members and interactions: `β_A = Σ_{p∈A} w_p + Σ_{p,q∈A} u_{pq}`, learning a per-species main effect `w_p` and pairwise synergy `u_{pq}`. This is a factorisation-machine view and it generalises to unseen teams built from seen parts — the property we need, since we will score *hypothetical* teams.
- **Matchup structure (advantage cycles).** Pokémon has rock–paper–scissors matchups: strength is not fully captured by a single scalar. The **Blade–Chest** and low-rank bilinear extensions add a vector per team and score `β_A − β_B + ⟨c_A, m_B⟩ − ⟨c_B, m_A⟩`, which can represent non-transitive advantage (A beats B beats C beats A). Including this term is what lets the model express "this team preys on the current meta" rather than "this team is generically strong."

4.3 Features

`x` should be a fixed-length encoding of a team invariant to Pokémon order: a multi-hot species vector (dimension = legal roster), augmented with observed-set summaries (lead rate, common item, revealed moves aggregated from ABRA's `sets` field), archetype tags (weather, Trick Room, Tailwind), and speed-tier and type-coverage summary statistics. A neural

encoder (a small permutation-invariant DeepSets/Transformer over the six member embeddings) is the expressive upgrade once the linear model saturates.

4.3.1 CHOMP's threat scoring as damage-grounded features

Species identity alone is a weak feature — two teams with the same six can play very differently, and what actually decides a matchup is *who KOs whom*. CHOMP already computes exactly this, and its output is the strongest feature JOLTEON can eat. CHOMP scores threats with the real Gen-9 damage pipeline on the Champions SP stat system and reports, for an attacker–defender pair, **pKO** — the fraction of the 16 damage rolls (85–100%) that knock out — combined with a speed check (does the attacker move first). Its bring-4 chooses the four that maximise KO-pressure/coverage across the opponent's six.

For JOLTEON, build the **coverage matrix** `K` where `K_{pq} = pKO(p → q)` over every attacker `p` in team A and defender `q` in team B (and its transpose for B→A), plus the speed-order mask. Summaries of `K` — how many of B's six A can OHKO, the best-case and worst-case coverage, symmetric speed control — become the features that carry most of JOLTEON's signal, because they are the real mechanics of the matchup rather than a usage proxy. This is grey-box modelling: CHOMP supplies the physics of a single exchange; JOLTEON learns how those exchanges aggregate into a game outcome across thousands of real results. CHOMP's own validation (its white paper, validation report, and referee report) is therefore part of JOLTEON's foundation — the feature generator is already tested.

4.4 Estimation and calibration

Maximise the regularised conditional log-likelihood:

```
ψ* = argmax_ψ  Σ_g [ y_g log Pψ(x_{A,g}, x_{B,g}) + (1−y_g) log(1 − Pψ(·)) ] − λ‖ψ‖².
```

A win **probability** is only useful if it is *calibrated* — when the model says 70%, the team should win ~70% of the time. We measure calibration with the

Brier score `(1/N) Σ (Pψ − y)²` and reliability diagrams, and correct residual miscalibration with **Platt scaling** or isotonic regression on a held-out split. Calibration, not raw accuracy, is what the team optimiser consumes, because it integrates these probabilities over the meta.

4.5 What tier 1 can and cannot do

It answers "does team A tend to beat team B?" from data, generalises to unseen teams via compositional features, and runs fast enough to score a whole search. It does **not** know *why* — it has no notion of turns, and it cannot tell you the line of play. It is a value function without a model. That is exactly enough for the outer optimisation loop and nothing more.

5. Tier 3 — SLOWKING, learning the dynamics (the true "reconstruction")

Tier 3 is what "reconstruct the simulator" literally means: learn the transition kernel `Tθ(s' | s, a)` and a policy/value on top, from logged transitions.

5.1 The dynamics model

Each replay is a sequence `(s_0, a_0, s_1, a_1, …, s_H, r)`. Every consecutive triple `(s_t, a_t, s_{t+1})` is a supervised example for a **world model**. Fit by maximum likelihood:

```
θ* = argmax_θ  Σ_{t}  log Tθ(s_{t+1} | s_t, a_t).
```

Because the true kernel factorises (each active Pokémon's move resolves largely independently given speed order), a structured model predicting damage-roll distributions, status procs, and stat changes per move is far more sample-efficient than a monolithic next-state predictor. Damage is *already* known in closed form (CHOMP's engine); the learned part is the residual — accuracy, secondary effects, switching outcomes, and the opponent's hidden set. This is a **grey-box** system identification problem: known physics plus a learned residual, which needs far less data than a black-box learner.

5.2 Partial observability

We never observe `s`; we observe `o` and must maintain a **belief** `b_t = Pr(s_t | o_{≤t}, a_{<t})`. The opponent's hidden set is the main latent variable. ABRA's usage model supplies the **prior** `Pr(set | species)` (how the ladder builds each Pokémon), and each revealed move/item is a Bayesian update. A particle filter over sampled opponent sets, each rolled forward through `Tθ`, is the standard tractable representation. Note the clean division of labour: ABRA (data) supplies the prior; the dynamics model supplies the update.

5.3 Solving the game on the learned model — search over beliefs, not states

With `Tθ` and beliefs, the game must be searched. Because the information is imperfect, the correct object is **not** a tree over states but a search over **public belief states (PBS)** — distributions over what each side could hold given everything commonly observed. This is the central idea of **ReBeL** (Recursive Belief-based Learning), which combines deep RL with search over PBSs and *provably converges to a Nash equilibrium* in two-player zero-sum imperfect-information games; in the perfect-information limit it reduces to AlphaZero. **Player of Games** and **Student of Games** later unified perfect- and imperfect-information search into one algorithm. This family — RL + search over beliefs — is the principled target for a Pokémon solver, and it subsumes the two simpler options: a per-turn **matrix-game Nash** (a small linear program at each node) with **minimax/expectimax** over joint actions, which is exactly PokéChamp's architecture (LLM samples actions, models the opponent, estimates leaf values; Elo 1300–1500); and the search-free route of a **policy/value network learned offline**, as Metamon does via behavioural cloning plus offline RL and AlphaStar Unplugged does at scale.

5.4 Why offline RL is hard here (and how the literature copes)

Learning a policy or value purely from logs suffers **distributional shift**: the model is queried at state–action pairs the logging players never took, where its estimates are unconstrained and optimistic. The canonical fixes are **conservatism** — Conservative Q-Learning (CQL) penalises out-of-

distribution action values; Implicit Q-Learning (IQL) avoids querying unseen actions entirely — and **return-conditioned supervised learning** — the Decision Transformer casts offline RL as sequence modelling, which is the family Metamon uses. Off-policy **evaluation** (weighted importance sampling, doubly-robust estimators) lets us score a policy from logs without deploying it, with variance that grows with policy divergence. The blunt honest point: without environment access, every tier-3 estimate is an extrapolation, and its trustworthiness must be *measured*, not assumed.

5.5 You do not simulate every path — the policy prunes the tree

The intuition that "*you would not simulate all the paths, machine learning just chooses the few viable ones*" is exactly how modern game AI is made tractable, and it has a precise form. The branching factor of a turn is large (each of two actives can move-with-target or switch), and the tree explodes geometrically with depth. AlphaZero tames this with a **policy prior** that biases the search toward promising actions; for genuinely large action spaces the fix is to **sample** rather than enumerate: **Sampled MuZero** expands only a subset of actions drawn from the prior, **AlphaStar** restricts StarCraft's enormous action space to a handful of policy-sampled actions per node, and **Gumbel MuZero** makes this sampling near-optimal with few simulations. PokéChamp is the Pokémon instance: the LLM proposes only the plausible moves, so the minimax tree never touches the overwhelming majority of legal-but-pointless lines.

For ABRA this means the learned policy is not a luxury on top of the simulator — it *is* what makes the simulator usable. A well-trained policy assigns almost all probability to the two or three viable options in a position, so search explores a tree with an effective branching factor of a few, not a few dozen. The cost of a "near-perfect" playout is therefore governed by the *policy's* quality, not by the raw size of the game tree. This also answers a practical worry: we never need to model every interaction explicitly by hand — a policy trained on enough real games learns which interactions matter and silently prunes the rest.

6. One model, many queries — playing and testing teams from a shared core

Is the team-tester the same model as the game-simulator? They should share a core. The AlphaZero / MuZero pattern is a single network with a shared representation and two heads — a **value** head `Vφ(belief)` and a **policy** head `πφ(action | belief)`. That one core answers both of ABRA's questions:

- **Testing teams against the meta** is the value head, marginalised over play and over the meta distribution: `E_{o~D}[ Vφ(team, o) ]` (§7). No game needs to be rolled out turn-by-turn to *rank* teams if the value head is calibrated — this is precisely tier 1, and it can be **distilled** from the full model into a fast outcome head so that team search stays cheap.
- **Giving the player the line** is the policy head plus search: from the current belief, the model returns the best move(s) — the very thing the coach surfaces. Because the model has learned how the mechanics and interactions resolve, it can hand the player a *recommended path*, not just a verdict. Every uploaded game both trains this core and is a state it can now advise on.

So the honest architecture is **one learned world/value/policy core, queried three ways** — as a fast team-evaluator (distilled value head), as a play-advisor (policy + belief search), and as the coach's explanation layer. Whether tier 1 remains a separate lightweight model or becomes a distilled head of the tier-3 core is an engineering choice, not a conceptual one; both are the same object viewed at different cost.

6.1 Continual learning — the model updates with every upload

"With every game upload the model improves" is **continual (online) learning**, and it has a known failure mode — **catastrophic forgetting**, where new data overwrites old competence. The standard guards are a **replay buffer** (keep training on a mix of old and new games, never new

alone), periodic re-fitting rather than pure online updates, and continuing to ingest the *whole* public ladder so the model does not narrow to only the games the current policy likes. Under these guards the flywheel (§8) is a continual-learning loop whose data distribution is steered, deliberately, toward the positions the player actually reaches.

7. Tier 2 — MEDICHAM, the pragmatic middle

MEDICHAM sidesteps learning dynamics by *specifying* them: it uses CHOMP's exact damage engine — the same pKO-over-16-rolls threat scoring as tier 1 — as a hand-built `T` for the dominant effect (damage), pair it with cheap policies (best-damage heuristics, or behaviour-cloned move priors from ABRA's `sets`), and roll a matchup out a few turns with a handful of stochastic samples to average over damage rolls. It is grounded in real math, requires no model training, and reuses code that already exists and is already tested. It is the right first *playout* model precisely because its errors are known and bounded (it ignores multi-turn tactics), and it is a strong baseline against which any learned tier-3 model must justify its cost.

8. DITTO — team optimisation, the outer loop

Given any tier as an evaluator `Ê[win | team]`, optimise the team. Formally, choosing a team `t` to maximise expected win rate against the **meta distribution** `D` (which ABRA measures):

```
t* = argmax_{t ∈ legal}  E_{o ~ D} [ Pwin(t, o) ].
```

The legal team space is astronomically large (hundreds of species choose six, times sets), so exhaustive search is out. The tractable families:

- **Evolutionary / local search.** Seed from meta archetypes or a user pool, mutate one slot at a time, keep improvements. Cheap, anytime, and matches how humans iterate.
- **Bayesian optimisation / bandits.** Treat team evaluation as expensive-noisy; use a surrogate over the team-feature space and an

acquisition rule to pick the next team to evaluate; **UCB**-style regret bounds apply. Natural when tier-2/3 evaluations are costly.

- **Game-theoretic (the principled version).** The meta is not fixed while you optimise — if you find a team that beats the field, the field adapts. The right object is a **Nash of the team-selection metagame**, approximated by **double-oracle / PSRO** (Policy-Space Response Oracles): alternately compute a best-response team to the current population and add it, growing toward equilibrium. This is exactly the "iterate on itself" the flywheel calls for, and it is the game-theoretic baseline VGC-Bench implements.

Coarse-to-fine ties the tiers together: tier 1 ranks thousands of candidate teams, tier 2 re-ranks the top few hundred, tier 3 vets the handful of finalists. No expensive evaluation is ever spent on an obviously bad team.

9. The self-improving flywheel, formalised

The five ABRA stages form a fixed-point iteration. Let `D_k` be the meta distribution at round `k`, `M_k` the model fit on data up to `k`, and `Π_k` the optimised team(s). Then:

```
M_k   = Fit(Data_k)                       # learn the model from logged
games
Π_k   = Optimise(M_k, D_k)                # best team(s) vs the measured
meta
Data_{k+1} = Data_k ∪ Play(Π_k)           # play; ingest every opponent
faced
D_{k+1}    = Meta(Data_{k+1})             # re-measure the meta
```

This is **expert iteration** — the AlphaZero pattern — with the environment supplied by the live ladder rather than self-play. Its appeal is compounding: each turn of the crank adds data where the current model is *most uncertain* (the games you actually play), which is the most valuable data to acquire. Its dangers are equally real and must be designed against: **feedback loops** (optimising against a meta you also perturb), **distributional collapse** (a model that only ever sees its own preferred games forgets the rest of the ladder — mitigated by continuing to ingest the *whole* public ladder, not only

your games), and **non-stationarity** (the meta drifts under you; models must be re-fit, not frozen). The flywheel is powerful *because* it is a control loop, and it must be treated with a control loop's caution.

10. Related work (grounding, 2024–2025)

- **PokéChamp** (Karten, Nguyen, Jin; ICML 2025 spotlight). LLM-augmented **minimax** tree search: the LLM supplies action sampling, opponent modelling, and value estimation inside a two-player game tree; no extra training; Elo 1300–1500 (top 10–30% of humans); ships the largest real-player dataset (3M+ games, 500k+ high-Elo). Directly validates tier-3-by-search (§5.3).
- **Metamon** (UT Austin RPL; RLC 2025). **Offline RL** with transformers on 5M+ trajectories reconstructed from human Showdown replays plus 20M+ self-play — the offline, learn-from-logs setting that is ABRA's exactly. Validates tier-3-by-learned-policy (§5.4).
- **VGC-Bench** (Angliss et al., 2025). A benchmark for *VGC specifically*, with the combinatorial team-strategy generalisation problem front and centre, and baselines spanning **behaviour cloning, multi-agent RL (PPO), game-theoretic (double-oracle), LLM, and heuristics** — the method menu of §5 and §7. Also extends poke-env with PettingZoo/VGC support.
- **PokéAgent Challenge** (NeurIPS 2025). Establishes Pokémon as a standard testbed for sequential decision-making under uncertainty, with Metamon and PokéChamp as baselines.
- **Classical foundations.** Bradley–Terry (1952) and Elo for paired comparisons (§4.2); CFR / Deep CFR for imperfect-information equilibria (§2); Dreamer and MuZero for learned-model planning (§5); CQL, IQL, Decision Transformer for offline RL (§5.4); PSRO / double-oracle for empirical-game equilibria (§7).

The takeaway: none of the three tiers is speculative. Each corresponds to a demonstrated method in the current literature. What ABRA contributes is not a new algorithm but the **data pipeline and the flywheel** that make these

methods self-sustaining on a live, closed-engine format.

11. Evaluation protocol

Every tier is scored on **held-out games** — a temporal split (train on older games, test on newer), never random, to respect meta drift and avoid leakage.

- **Outcome model (tier 1).** Test log-likelihood, **Brier score**, calibration reliability diagram, and AUC. Report against two baselines: the constant 50% predictor, and the Bradley–Terry main-effects-only model. A model that cannot beat "the higher-usage team wins" is not yet useful.
- **Dynamics model (tier 3).** Per-step next-state log-likelihood and long-horizon rollout divergence (does a 10-turn rollout stay near reality?). Policy quality by **off-policy evaluation** and, ultimately, by **live ladder Elo**, the only unarguable metric.
- **The honest ceiling.** Pokémon has irreducible variance: imperfect information plus damage rolls, accuracy, and speed ties mean even a perfect model cannot predict single games with high accuracy — which is why the strongest published agents plateau around Elo 1300–1500 rather than dominating. A prior internal analysis on a coverage heuristic found game-level correlation ≈ 0; that finding stands as the caution. We therefore state success as **calibrated aggregate win-rate improvement of optimised teams over a baseline**, measured over many games, not per-game oracular accuracy.

12. Feasibility — can we actually build this?

Yes, tier by tier, with honesty about each.

- **Tier 1 (JOLTEON) is built.** A Bradley–Terry logistic model with a min-sample species floor, trained on 5,000 real ladder games (temporal split, humans only). Measured result: **56.6% held-out accuracy vs 49.6% for a coin flip**, Brier 0.251 (calibrated). Modest, exactly as this

domain's variance predicts, but genuinely above chance from team composition alone — and it improves as the flywheel adds games and once CHOMP's coverage features (§4.3.1) replace bare species identity. Antisymmetry and calibration are pinned by `tests/test-jolteon.py`. This validates the tier-1 path empirically, not just on paper.

- **Tier 2 reuses CHOMP.** The damage engine and matchup evaluation exist and are tested; tier 2 is a rollout harness plus behaviour-cloned move priors from ABRA's `sets`. Low new risk.
- **Tier 3 is a research effort.** A learned dynamics model, belief tracking, and equilibrium search (or offline-RL policy) is the frontier the cited papers occupy. It is *possible* — they prove it — but it needs real compute, careful offline-RL practice, and sustained work, and it should be judged against the tier-2 baseline before its cost is accepted. Grey-box structure (§5.1) and a warm start from open Showdown data are the levers that make it tractable rather than hopeless.

The disciplined path is to ship tier 1, keep the flywheel turning so the dataset compounds, add tier 2 for finalist vetting, and treat tier 3 as a funded research track whose bar is set by tier 2.

13. References

1. Karten, S., Nguyen, A. L., Jin, C. (2025). *PokéChamp: an Expert-level Minimax Language Agent.* ICML 2025 (spotlight). arXiv:2503.04094.
2. UT Austin RPL (2025). *Metamon: Human-Level Competitive Pokémon via Scalable Offline Reinforcement Learning with Transformers.* RLC 2025. arXiv:2504.04395.
3. Angliss, C. et al. (2025). *VGC-Bench: A Benchmark for Generalizing Across Diverse Team Strategies in Competitive Pokémon.* arXiv:2506.10326.
4. *PokéAgent Challenge*, NeurIPS 2025.
5. Bradley, R. A., Terry, M. E. (1952). *Rank Analysis of Incomplete Block Designs.* Biometrika.

6. Zinkevich, M. et al. (2007). *Regret Minimization in Games with Incomplete Information (CFR).* NIPS.
7. Kumar, A. et al. (2020). *Conservative Q-Learning for Offline RL.* NeurIPS.
8. Kostrikov, I. et al. (2021). *Offline RL with Implicit Q-Learning (IQL).*
9. Chen, L. et al. (2021). *Decision Transformer: RL via Sequence Modeling.* NeurIPS.
10. Schrittwieser, J. et al. (2020). *Mastering Atari, Go, Chess and Shogi by Planning with a Learned Model (MuZero).* Nature.
11. Lanctot, M. et al. (2017). *A Unified Game-Theoretic Approach to Multiagent RL (PSRO).* NeurIPS.
12. Chen, S., Joachims, T. (2016). *Modeling Intransitivity in Matchup Outcomes (Blade–Chest).* ICML.
13. Brown, N. et al. (2020). *Combining Deep RL and Search for Imperfect-Information Games (ReBeL).* NeurIPS.
14. Schmid, M. et al. (2021/2023). *Player of Games* / *Student of Games:* unified search + learning across perfect- and imperfect-information games. Science (2023).
15. Hubert, T. et al. (2021). *Learning and Planning in Complex Action Spaces (Sampled MuZero).* ICML.
16. Danihelka, I. et al. (2022). *Policy Improvement by Planning with Gumbel (Gumbel MuZero).* ICLR.
17. Vinyals, O. et al. (2019). *Grandmaster level in StarCraft II (AlphaStar).* Nature; and *AlphaStar Unplugged* (2023), large-scale offline RL.
18. French, R. (1999). *Catastrophic Forgetting in Connectionist Networks.* Trends in Cognitive Sci.
19. Hooper, W. (2026). *CHOMP white paper, validation report, and referee report* — the Gen-9 damage pipeline, the pKO-over-16-rolls threat score, and the coverage-based bring-4, used here as JOLTEON's feature generator and MEDICHAM's dynamics. `Pokemon/CHOMP/docs/`.

Companion documents. ABRA white paper · Executive summary · Technical documentation